



INTERNAL ENGINEER

Backend – Bill Generation

ion Guide

 For: Development Intern  Store

y  Sep 2026

1 Context & Your Task

The SnapMart demo database is live on **Neon PostgreSQL**. It contains **547 unique sellable SKUs** across 30 subcategories, with full pricing, inventory, and GST data already computed and stored. You have access to the database connection string (API key).

Your task: Wire the application's backend to retrieve the correct data from the database and populate the customer bill. Specifically:

- Retrieve `gst_amount` from `product_pricing` per scanned item
- Sum it across all cart items to show as **"GST & Other Charges"** on the bill
- Apply a fixed **Service Fee** that is hardcoded in the app — not stored in the database

2 Database Structure — `product_pricing` Table

The `product_pricing` table holds one record per SKU per store. Every GST column is pre-computed and stored — you do not need to calculate anything in the backend.

Column	Type	Example Value	Purpose	Expose to App?
<code>final_price</code>	NUMERIC(10,2)	₹168.00	Price customer actually pays (GST-inclusive)	✓ YES
<code>gst_amount</code>	NUMERIC(10,2)	₹25.63	Total GST embedded in <code>final_price</code> → use this for bill	✓ YES – BILL
<code>gst_rate</code>	NUMERIC(5,2)	18.00	GST slab percentage (0 / 5 / 12 / 18 / 28)	OPTIONAL
<code>taxable_value</code>	NUMERIC(10,2)	₹142.37	Price before GST (for compliance & accounting)	✗ INTERNAL ONLY
<code>cgst_rate</code>	NUMERIC(5,2)	9.00	Central GST rate ($\text{gst_rate} \div 2$)	✗ INTERNAL ONLY
<code>cgst_amount</code>	NUMERIC(10,2)	₹12.82	Central GST amount (for GSTR filing)	✗ INTERNAL ONLY
<code>sgst_rate</code>	NUMERIC(5,2)	9.00	State GST rate — Tamil Nadu ($\text{gst_rate} \div 2$)	✗ INTERNAL ONLY
<code>sgst_amount</code>	NUMERIC(10,2)	₹12.81	State GST amount (for GSTR filing)	✗ INTERNAL ONLY
<code>gst_hsn_code</code>	VARCHAR(20)	3305	HSN code for govt. compliance invoices	✗ INTERNAL ONLY

⚠ Important – Why do we keep all those columns if we only show one?

Indian GST law requires businesses to maintain CGST, SGST, taxable value, and HSN records for every transaction for GSTR-1 / GSTR-3B government filings. These columns exist for **compliance and accounting**. The customer-facing app only surfaces `gst_amount`. Never delete those columns.

3 Understanding GST in Indian Retail (Read This Carefully)

Key rule: In India, the MRP printed on every product is *always GST-inclusive*. The customer never pays GST on top of the price — it is already inside the price.

EXAMPLE — DOVE SHAMPOO 180ML (18% GST SLAB)

```
-- What is stored in the database:
final_price    = ₹168.00    ← what customer pays (MRP)
taxable_value  = ₹142.37    ← base price before GST  [142.37 × 1.18 = 168.00]
gst_amount     = ₹25.63     ← GST already inside ₹168  [168 - 142.37]
cgst_amount    = ₹12.82     ← Central GST 9%  (goes to Centre)
sgst_amount    = ₹12.81     ← State GST 9%   (goes to Tamil Nadu)

-- The bill shows GST as an INFORMATIONAL line, not an extra charge:
```

Item Total: ₹168.00 ← customer pays this
of which GST: ₹25.63 ← already inside ₹168 – shown for transparency

4 Bill Structure — What to Show & Where It Comes From

Payment Summary	Source
Item Total SUM(final_price × qty) for all cart items	₹520.00 → product_pricing.final_price
Service Fees Fixed by business – hardcoded in app config	₹40.00 → App constant (NOT in DB)
Snap Up Discount SUM(discount_value × qty) or promo code	– ₹40.00 → product_pricing.discount_value
GST & Other Charges SUM(gst_amount × qty) – already inside Item Total	₹17.00 → product_pricing.gst_amount ✓
Total to Pay	₹537.00

Note on the math: Total to Pay = Item Total + Service Fees – Discount. GST is *shown* on the bill but is already embedded inside Item Total — it is not added again. The sum (520 + 40 – 40 = 520, then +17 shown separately) is displayed for transparency per regulatory guidelines.

5 The Exact Database Query to Write

When the customer scans a product barcode, call this parameterised query on the server. Return only these columns to the frontend — nothing else.

```
SQL — PER-ITEM DATA FOR CART (NODE.JS / PG)
-- Run this for each scanned barcode
SELECT
  s.barcode,
  s.variant_name,
  s.quantity_value,
  s.quantity_unit,
  b.name          AS brand,
  pp.selling_price,
  pp.discount_type,
  pp.discount_value,
  pp.final_price,
  pp.gst_amount,  -- ← THIS is "GST & Other Charges" contribution per item
  pp.gst_rate,    -- optional: show "18%" badge on the item
  inv.available_qty,
  inv.stock_status
FROM product_pricing pp
JOIN skus s          ON s.id = pp.sku_id
JOIN products p      ON p.id = s.product_id
JOIN brands b        ON b.id = p.brand_id
JOIN inventory inv    ON inv.sku_id = s.id AND inv.store_id = pp.store_id
```

```
WHERE s.barcode    = $1  -- scanned barcode e.g. 'SNAP0000000042'
AND pp.store_id   = $2; -- store UUID from session
```

NODE.JS — BILL TOTAL CALCULATION (BACKEND, NOT FRONTEND)

```
// app-level constants (NOT in database)
const SERVICE_FEE = 40.00; // ← fixed by business – change here only

// cartItems = array of { final_price, discount_value, gst_amount, qty } from DB
function calculateBill(cartItems) {
  const itemTotal = cartItems.reduce((s, i) => s + i.final_price * i.qty, 0);
  const discount  = cartItems.reduce((s, i) => s + i.discount_value * i.qty, 0);
  const gstTotal  = cartItems.reduce((s, i) => s + i.gst_amount * i.qty, 0);
  const totalToPay = itemTotal + SERVICE_FEE - discount;

  return {
    item_total:      round2(itemTotal),
    service_fee:     SERVICE_FEE,        // hardcoded
    snap_up_discount: round2(discount),
    gst_and_charges:  round2(gstTotal),   // ← "GST & Other Charges" on bill
    total_to_pay:     round2(totalToPay),
  };
}

const round2 = n => Math.round(n * 100) / 100;
```

6 Rules — What You Must NOT Do

❌ Do not expose to frontend:

- cgst_amount
- sgst_amount
- cgst_rate
- sgst_rate
- taxable_value
- gst_hsn_code
- DATABASE_URL / credentials

✅ Safe to expose to frontend:

- variant_name
- brand
- final_price
- selling_price
- discount_value
- gst_amount (as GST & Other Charges)
- gst_rate (optional badge)
- available_qty / stock_status

❌ **Never store Service Fee in the database.** Service Fee is a business rule, not a product attribute. If the fee changes (₹40 → ₹50), you update *one constant* in the app config — not a database row. Storing it in the database would mean re-querying it per order and creating unnecessary complexity.

7 Quick Reference — Bill Field to DB Column

Bill Line	Value Comes From	Location
Item Total	SUM(product_pricing.final_price × qty)	DATABASE
Service Fees	const SERVICE_FEE = 40.00	APP CONFIG
Snap Up Discount	SUM(product_pricing.discount_value × qty)	DATABASE
GST & Other Charges	SUM(product_pricing.gst_amount × qty)	DATABASE ✓
Total to Pay	Item Total + Service Fee - Discount	BACKEND CALC

🔑 Database Access

You have been given the Neon PostgreSQL connection string (DATABASE_URL). Rules for using it:

- Store it in a .env file — never commit it to Git
- Access the database **only from the server/backend** — never from a browser or mobile client
- Always use **parameterised queries** (use \$1, \$2 placeholders) — never string-concatenate user input into SQL
- The SSL mode is require — do not disable it
- Use the **pooler** endpoint (the URL already includes -pooler in the hostname) for the application server

